# On-Device Continual Personalization for RSS Readers: A TinyML Approach to Privacy-Preserving Recommendation

May 15, 2026

**Abstract**

This draft proposes an on-device recommender framework for RSS reading that performs continual personalization using only local user behavior. The objective is to mitigate model collapse and privacy risks by keeping interaction logs, feature extraction, adaptation, and model updates entirely on-device. The work targets Android first and is designed to remain compatible with future TinyML transfer constraints for STM32-class hardware. The current implementation path prioritizes a deployable local baseline with strict efficiency constraints on latency, memory footprint, storage growth, and battery cost.

## Contents

# 1 Introduction

## 1.1 Problem Statement

TODO: Define the practical and scientific problem in RSS recommendation under privacy and resource constraints.

## 1.2 Motivation

TODO: Explain why local-only continual learning is needed and where cloud-first recommenders fail for this context.

## 1.3 Contributions

TODO: Add concrete contributions after first stable experiment cycle.

# 2 Background and State of the Art

## 2.1 On-Device Recommender Systems

TODO: Summarize key approaches from the DeviceRS survey.

## 2.2 Continual Learning at the Edge

TODO: Summarize edge continual training assumptions and limits.

## 2.3 TinyML Online Learning

TODO: Extract STM32-relevant constraints and lessons from TinyOL.

## 2.4 Privacy-by-Design in Mobile Systems

TODO: Map privacy principles to this architecture.

# 3 Research Gap and Novelty

## 3.1 Gaps in Related Work

TODO: Fill from matrix in related-work-matrix.md.

## 3.2 Proposed Novelty

TODO: Define novelty claims and boundary conditions.

## 3.3 Deployment-Centric Claims

TODO: Formalize efficiency-first claims with measurable thresholds for inference latency, online update overhead, memory footprint, and battery impact.

# 4 Methodology

## 4.1 System Architecture

TODO: Add architecture figure and component breakdown.

### 4.2 User Behavior Model

Based on empirical observation of real user interactions with RSS readers, we identify three distinct engagement stages:

- **Stage 1 (Feed Tab):** Browsing list of articles with filtering (all/read/unread), sorting (date), search, and implicit scroll-based item exposure. Users spend time proportional to subject interest, title characteristics, and content language.
- **Stage 2 (Article Preview):** Decision point where user views article snippet (title, image, description) and chooses an action: open in browser, read in app, save for later, share, or return.
- **Stage 3 (Reader):** Full article consumption with available features: translate, read-aloud, notes, bookmark, share, and save. Reading speed and dwell time correlate with character count, language, and content quality.

, stage-transition, search Behavioral signals extracted at each stage inform the continual learner. Stage transitions and feature usage (e.g., translation, read-aloud, annotations) serve as learning labels and engagement proxies.

### 4.3 Signal Extraction Schema

- **Impressions:** Article rank, filter, sort order, title length, image presence, detected language.
- **Opens:** Action type (preview, read-in-app, browser), article metadata.
- **Read sessions:** Dwell time, max scroll depth, feature usage (translate, read-aloud, notes), content language and length.
- **Search:** Query string for intent inference (future work).
- **Stage transitions:** Entry/exit timestamps and dwell per stage.

### 4.4 Efficiency Cascade: Joint Optimization of Architecture, Quantization, and Learning

The research strategy centers on a three-tier efficiency pipeline designed to make local continual learning feasible on resource-constrained mobile and embedded platforms:

1. **Architecture Search (µNAS):** Automated discovery of minimal-footprint models under strict phone/microcontroller latency and memory budgets. Reframe RSS recommendation as lightweight decision operations (topic scoring, ranking) rather than end-to-end black-box models.
2. **Weight Quantization (BNN):** Binarization of learned weights and activations reduces model size by 8-32x and speeds inference on low-power hardware. Evaluate utility loss (ranking quality) versus compression gain (storage, latency, power).
3. **Continual Learning (CL + TinyOL):** On-device incremental adaptation of the quantized model using bounded replay and drift-aware scheduling. Target $< 100$ms update latency per user event.

**Expected Outcome:** Smartphone recommendation model below 5MB, inference below 50ms, update below 100ms, with validated transfer to STM32 microcontroller.

### 4.5 Selected Implementation Approach (Current)

- **Phase A (current):** Lightweight local continual learner with event-driven updates and bounded replay, fully on-device.
- **Phase B (near-term):** Constrained architecture search (µNAS-inspired) combined with binarized weight quantization (BNN) to reduce model footprint 8-32x. Validate on-device inference latency and update cost under phone constraints.
- **Phase C (research goal):** TinyML transfer track with compact model/state representation aligned with STM32 constraints. Cross-device evaluation to validate generalization.

### 4.6 Local Interaction Pipeline

The local interaction pipeline is currently implemented as an event log with bounded replay capacity and explicit retention controls.

- **Event families:** `impression`, `open`, `read_session`, `search`, and `stage_transition`.
- **Session enrichment:** Reader sessions store dwell time, max scroll depth, completion signal, and feature usage flags (translation, read-aloud, note-taking).
- **Bounded storage:** Event buffer is capped to 5000 records to prevent unbounded growth.
- **Retention policy:** Time-based pruning is supported (default 30 days; configurable to longer windows in settings).
- **Cursor-based replay:** Continual updates process only unseen events via persistent cursor, reducing repeated compute overhead.

### 4.7 Continual Learning Strategy

The baseline learner is an event-driven additive updater over topic-specific preference weights.

- **Signal-to-update mapping:** opens provide positive intent signal; read sessions add dwell-depth-completion weighted reinforcement.
- **Replay model:** Incremental cursor replay applies updates only to newly recorded events since last step.
- **Stability guardrails:** Topic weights are clamped to bounded range to avoid runaway growth and collapse.
- **Drift detector (implemented):** For each topic, short-window and long-window EMAs of update deltas are tracked; divergence is computed as $|EMA_{short} - EMA_{long}|$.
- **Drift alerting:** Topics whose divergence exceeds threshold after minimum evidence are marked as drift-flagged for downstream ranking/scheduling policies.

### 4.8 Efficiency and Footprint Constraints

TODO: Report budgeted constraints and observed values:

- Inference latency per ranked item and per list refresh.
- Local update latency and scheduling frequency.
- Memory and persistent storage footprint growth over time.
- Battery and thermal overhead in realistic usage sessions.

### 4.9 Privacy Model

TODO: Threat model and guarantees for local-only design.

## 5 Experimental Protocol

### 5.1 Research Questions and Hypotheses

TODO: Add RQ1–RQn and corresponding hypotheses.

### 5.2 Datasets and Real-Usage Collection

TODO: Describe realistic data collection without mock data.

### 5.3 Baselines and Ablations

TODO: Static baseline vs CL-v1 and ablations.

### 5.4 Metrics

TODO: Utility, diversity, drift resilience, energy, memory, and latency.

### 5.5 Primary Evaluation Targets

- Accuracy and ranking quality under non-stationary user behavior.
- Adaptation speed after concept drift events.
- Resource efficiency: latency, memory footprint, storage overhead, energy cost.
- Privacy robustness under local-only constraints and no raw behavior export.

## 6 Preliminary Implementation Status

- v2.0.3: Added on-device drift detector using short-vs-long EMA divergence per topic with thresholded topic-level flags.
- v2.0.2: Enhanced event schema to capture stage transitions, feature usage (translate, read-aloud, notes), and article metadata (language, title length, content length).
- v2.0.2: Integrated user behavior model from empirical observation into signal extraction.
- v2.0.2: Added strategy for efficiency cascade: µNAS (architecture search) + BNN (weight quantization) + CL (continual learning).
- v2.0.1: Added local on-device interaction event pipeline in app code.
- v2.0.1: Added feed-level impression/open logging and reader dwell-depth session logging.
- v2.0.1: Added settings controls for local learning enablement, retention, summary, and reset.

## 7 Threats to Validity

TODO: Internal, external, construct, and conclusion validity.

## 8 Ethics and Privacy Considerations

TODO: User consent, transparency, and local data governance.

## 9 Conclusion and Next Steps

TODO: Summarize expected outcomes and near-term milestones.